Rheinland-Pfälzische
Technische Universität
Kaiserslautern
Landau

Fachbereich Informatik

Master's Thesis

Creating and Benchmarking Learned Database Components using LLMs

Author:
Preksha Rajesh Shah

submitted on:
April 23, 2026

Supervisor
Angjela Davitkova

Reviewers
Prof. Dr.-Ing. Sebastian Michel

Abstract

Lorem ipsum

Zusammenfassung

Lorem ipsum

Contents

1 Introduction **1**
1.1 Problem Statement . . . 1
1.2 Proposed Approach . . . 1
1.3 Contributions . . . 2

2 Background **3**
2.1 Learned Database Systems and Cardinality Estimation . . . 3
2.2 MSCN and Deep Learning-based Estimation . . . 3
2.3 Workload Sensitivity and Challenges . . . 4
2.4 LLM-based SQL Generation . . . 5
2.5 Generative AI for Database Components . . . 5
2.6 Active Learning for Data Efficiency . . . 6
2.7 Data Diversity, Personas, and KL Divergence . . . 6
2.8 Local Model Hosting for Reproducibility . . . 7
2.9 Summary and Motivation . . . 7

3 Related Work **8**
3.1 Learned Cardinality Estimation . . . 8
3.2 Workload Characteristics and Model Generalization . . . 8
3.3 LLM-based SQL Workload Generation . . . 9
3.4 Generative AI for Training Database Components . . . 9
3.5 AL for Efficient Data Utilization . . . 10
3.6 Data Diversity and Distribution Alignment . . . 10
3.7 Positioning of the Proposed Approach . . . 10

4 Approach **12**
4.1 Benefits of the Proposed Framework . . . 12
4.2 System Architecture . . . 13
4.3 Pipeline Overview . . . 13
4.4 Summary . . . 14

5 Query Generation Approach **15**
5.1 LLM-Based Query Generation . . . 15
5.2 Prompt Design . . . 15
5.3 Diversity and Batch Generation . . . 17
5.4 Multi-Layer Validation Pipeline . . . 18
5.5 Synthetic Query Generator . . . 18
5.6 Summary . . . 20

6 Training Approach **21**
6.1 Database Labeling . . . 21
6.2 Feature Extraction and Bitmap Generation . . . 21
6.3 Multi-Set Convolutional Network . . . 22
6.4 Label Normalization and Loss Function . . . 24
6.5 Active Learning Framework . . . 25
6.6 Summary . . . 25

7 Experimental Evaluation **27**
7.1 Experimental Setup 27
7.2 Query Generation Analysis 27
7.3 Active Learning and Model Training Results 30
7.4 Labeling Efficiency Analysis 34
7.5 Comparison with PostgreSQL Optimizer 34
7.6 Pipeline Summary and Discussion 35

8 Conclusion **40**
8.1 Summary of Contributions 40
8.2 Impact and Implications 40
8.3 Limitations and Future Work 41
8.4 Final Remarks 41

List of Figures

4.1 High-level system architecture showing the end-to-end pipeline from schema ingestion through query generation, validation, labeling, and active learning to final model evaluation. The dashed feedback loop indicates iterative refinement through multiple active learning rounds. 13

5.1 Structure of the LLM prompt template, consisting of five hierarchically organized components that guide the model from context understanding to constrained output generation. 16

5.2 Multi-layer validation pipeline for LLM-generated queries. Each layer progressively filters invalid queries, with rejected queries logged for analysis. 19

6.1 Complete MSCN architecture. Each query component (tables, predicates, joins) passes through a dedicated two-layer neural network with dropout regularization, followed by average pooling. The pooled representations are combined and processed through an output layer to produce a normalized cardinality estimate $\hat{y} \in [0, 1]$. 23

7.1 Entity-relationship diagram of the IMDB database schema. The `title_principals` table acts as a central junction connecting titles to people, while `title_ratings` provides aggregate user feedback per title. These relationships define the valid join paths for generated queries. 28

7.2 Structural feature distributions across the generated workload. The balanced coverage of different table counts, join complexities, and predicate counts confirms that the diversity hint mechanism prevents the LLM from converging on a narrow subset of query patterns. 29

7.3 Query validation success rates across the four filtering layers. The high pass rate at each stage demonstrates the effectiveness of the prompt engineering constraints in guiding the LLM toward valid query structures. 29

7.4 Distribution of table count per query. The weighted mix of single-table, two-table, and three-table queries ensures coverage of both simple selections and complex multi-way joins. 30

7.5 Distribution of predicate count per query. The majority of queries contain one to three predicates, reflecting a realistic range of filter complexities. 30

7.6 Distribution of join conditions per query. Queries with zero joins correspond to single-table queries, while one and two join conditions correspond to two-table and three-table queries, respectively. 31

7.7 Distribution of true cardinalities ($\log_{10}$ scale). The wide range spanning several orders of magnitude forces the model to learn accurate estimates for both highly selective queries returning few rows and broad queries spanning millions of tuples. 31

7.8 Active learning curve showing validation median Q-error versus the number of labeled queries. The steep initial decline demonstrates rapid learning from the first few rounds, while the flattening tail indicates diminishing returns as the model approaches its capacity on the given data. 32
7.9 Training loss curves across active learning rounds. The consistently decreasing trajectory demonstrates stable convergence, with each round starting from a better baseline than the previous one due to the expanded and more informative training set. . . . 32
7.10 Predicted vs. actual cardinality on a log-log scale. The concentration of points along the diagonal indicates that the MSCN model produces well-calibrated estimates across the full cardinality range, from selective single-table queries to large multi-table joins. 33
7.11 Cumulative distribution function of Q-errors. The steep rise at low Q-error values indicates that the majority of queries are estimated within a factor of 2–5 of their true cardinality. 34
7.12 Q-error box plots per active learning round. The narrowing boxes and decreasing medians demonstrate how each round improves estimation accuracy. 34
7.13 Q-error statistics across active learning rounds. The consistent decrease in all percentile metrics confirms that active query selection improves not only typical-case accuracy but also tail-end performance on the most challenging queries. 35
7.14 Labeling efficiency comparison. The active learning approach achieves competitive accuracy with significantly fewer labeled queries than a fully supervised baseline, demonstrating that intelligent query selection reduces the dominant cost factor in the pipeline. 36
7.15 Labeling success rate across the pipeline. Queries that exceed the database timeout are assigned fallback cardinalities. The high success rate confirms that the generated queries are overwhelmingly executable within the configured time limits. 37
7.16 Q-error CDF comparing the MSCN model against PostgreSQL's native estimator. A curve shifted further left indicates better estimation accuracy. 37
7.17 Predicted vs. actual scatter comparing MSCN and PostgreSQL. Points closer to the diagonal represent more accurate estimates. . 37
7.18 Pipeline summary dashboard aggregating key metrics from all stages. This consolidated view enables rapid assessment of the entire experimental run, from data generation quality through model training convergence to final estimation accuracy. 38

List of Algorithms

1 LLM-Based Query Generation 17
2 Active Learning for Cardinality Estimation 26

List of Tables

5.1 Comparison of LLM-based and synthetic query generation approaches. 18

7.1 Experimental configuration parameters. 28

Chapter 1

Introduction

1.1 Problem Statement

Modern database systems rely heavily on cardinality estimation to optimize query execution. Accurate estimation of intermediate result sizes is essential for selecting efficient execution plans. However, traditional cardinality estimators are based on simplifying assumptions such as attribute independence and uniform data distributions, which rarely hold in real-world datasets. As a result, these estimators often produce inaccurate predictions, leading to suboptimal query plans and degraded system performance.

Recent advances in learned database systems have demonstrated that machine learning models can replace traditional components and achieve significantly improved estimation accuracy. In particular, the Multi-Set Convolutional Network (MSCN) shows that deep learning models can capture complex correlations across tables, joins, and predicates [4]. Despite these improvements, learned approaches introduce a new challenge: they require large volumes of labeled training data, where each label is obtained by executing a query on a database system. This process is computationally expensive, time-consuming, and difficult to scale.

At the same time, large language models (LLMs) have shown strong capabilities in generating structured data, particularly SQL queries. Frameworks such as SQLStorm demonstrate that LLMs can generate diverse and realistic SQL workloads at scale [11]. However, existing approaches still rely on executing these queries to obtain ground truth labels, leaving the fundamental bottleneck of execution-based labeling unresolved.

Therefore, a key open problem is how to generate large-scale, diverse, and labeled training data for learned database components without relying on expensive database execution.

1.2 Proposed Approach

In this thesis, we propose a novel perspective on the role of large language models in database systems. Instead of using LLMs during query execution, we treat them as data generators. Specifically, an LLM is used to generate synthetic SQL queries along with their corresponding labels or training signals. These query–label pairs are then used to train a supervised model that implements a database component, such as a learned cardinality estimator. The trained model, rather than the LLM, is used during runtime.

To address the challenge of data efficiency, we integrate active learning into the training process. Instead of labeling all generated queries, the model selectively

identifies the most informative queries based on uncertainty and diversity criteria. Only these selected queries are labeled and incorporated into the training dataset, significantly reducing the cost of data generation while maintaining high model performance.

The proposed approach is implemented as a fully automated, schema-agnostic pipeline that integrates query generation, validation, feature extraction, active learning, and model training. The pipeline supports both LLM-based and synthetic query generation and can operate on arbitrary database schemas without manual customization. This enables scalable experimentation and efficient training of learned database components.

Our approach is motivated by prior observations that workload diversity and query characteristics play a crucial role in the performance of learned models [12]. By leveraging LLMs to generate diverse workloads and combining this with active learning, we aim to improve the robustness and generalization of learned cardinality estimators while reducing reliance on execution-based labeling.

1.3 Contributions

The main contributions of this thesis are fourfold. First, we propose a novel framework that leverages large language models to generate synthetic SQL queries and their corresponding training signals, reducing dependence on manually collected query workloads. Second, we introduce an active learning-based training strategy for learned cardinality estimation, enabling efficient selection of informative queries and significantly reducing labeling costs. Third, we design and implement a fully automated, schema-agnostic pipeline that integrates query generation, validation, feature extraction, active learning, and model training into a unified system. Fourth, we demonstrate through extensive experiments on the IMDB benchmark that combining LLM-based data generation with active learning improves scalability, diversity, and effectiveness of training data for learned database components.

Chapter 2

Background

2.1 Learned Database Systems and Cardinality Estimation

Modern database systems use query optimizers to optimize queries for efficient execution. At the heart of query optimization is cardinality estimation. Cardinality estimation is used to predict the number of tuples that are generated during intermediate steps of query execution. The estimates are used to optimize queries and improve performance. However, small errors in cardinality estimation can lead to poor query optimization and performance. Conventional cardinality estimation techniques use statistical estimates such as histograms and sampling. Other techniques include making assumptions such as attribute independence and uniform distribution of values. Although conventional techniques are computationally efficient, they do not reflect real-world data characteristics. Real-world data has complex correlations among attributes and tables, especially for join-heavy queries. As such, conventional techniques often yield poor estimates for queries with multiple predicates and joins.

In addition to inaccuracies caused by independence assumptions, another limitation of traditional approaches lies in their inability to adapt dynamically to evolving data distributions. In real-world applications, data is continuously updated, and query workloads may change over time. Static statistical summaries often fail to capture these changes effectively, leading to outdated estimates and suboptimal execution plans. This problem becomes more severe in large-scale systems where maintaining accurate statistics is itself a costly operation. As a result, there has been a growing interest in approaches that can learn directly from data and adapt to changing workloads.

This limitation has motivated the exploration of learned database systems, where machine learning models replace traditional components. Instead of relying on predefined assumptions, these models learn patterns directly from data, enabling them to capture correlations that are otherwise difficult to model. Learned cardinality estimation is one of the most actively studied areas within this paradigm. These models aim to generalize across different query structures and data distributions, providing more robust and accurate estimates compared to traditional techniques.

2.2 MSCN and Deep Learning-based Estimation

One important advancement in the area of learning-based query cardinality estimation is the development of a new technique called Multi-Set Convolutional Network (MSCN), proposed by Kipf et al. in[4]. The technique reformulates query cardinality estimation as a supervised learning task, where query result

sizes are estimated based on query structures.

The basic idea behind the MSCN technique is that it represents queries in the form of sets. Unlike traditional methods, where a query is simply represented as a string of characters, MSCN represents a query as three different sets, namely tables, joins, and predicates. Each set is processed separately using shared layers of a neural network. The basic idea behind this is to make the query invariant to the ordering of query components, which is important because the semantics of a query do not depend on the order in which tables and predicates are listed.

The other key feature of the MSCN is its use of bitmap-based sampling. In the process, a subset of the tuples in a given base table is sampled, and the predicates are applied to the sample to produce bitmaps. The bitmaps then represent the query predicates that are satisfied, and this additional information helps the model to better account for the correlations between the predicates and the data.

In addition, the MSCN uses normalization to ensure that the cardinality estimates are within a certain range that can be learned by the model. This is crucial, especially considering that the cardinality estimates can differ considerably across queries.The MSCN uses loss functions that involve relative errors, as opposed to absolute errors, which is more appropriate for query optimization tasks.

Despite its strong performance, MSCN depends heavily on labeled training data. Each training example requires a query and its true cardinality, which is obtained by executing the query on a database. As query complexity increases, the cost of execution grows significantly, making it difficult to scale the training process. This limitation highlights a fundamental challenge in learned database systems: the high cost of data generation. In addition, the reliance on execution makes it difficult to quickly generate new training data when schemas or workloads change.

2.3 Workload Sensitivity and Challenges

The performance of learned cardinality estimators not only depends on the architecture used but also on the nature and characteristics of the training workload. The study conducted by Wang et al. [12] offers a comprehensive analysis on how different query properties influence the performance of learned cardinality estimators.

For example, a learned model may not perform well when it is tested on queries that have different types of predicates, such as range predicates, when it is only trained on queries that have equality predicates. Similarly, a query may have high selectivity and join complexity, causing a large estimation error. This example again emphasizes that the training data should be highly diverse.

Another important aspect to be considered is related to workload shift, where the query distribution is different from the training distribution. In this case, learned models may not perform well, and this again emphasizes that the training data should be highly diverse. Moreover, the interaction between multiple

query features, such as the number of joins and predicate selectivity, can further complicate the estimation process. These interactions are often difficult to model explicitly, which reinforces the importance of exposing models to a wide variety of query patterns during training.

2.4 LLM-based SQL Generation

Recently, large language models have been recognized as effective means for generating structured data. In this regard, the SQLStorm framework[11] has been presented to show that LLMs can be effectively used to produce large volumes of varied and realistic SQL queries. In this regard, the SQLStorm framework exploits the capabilities of schema-informed prompting to produce varied and realistic queries.

Unlike other query generation techniques that are based on template-based and rule-based approaches, LLMs can be effectively used to produce queries that can capture a wide range of query patterns. This is one of the main reasons that LLMs can be effectively used to produce training datasets for learning-based query generation techniques. In addition to this, LLMs can be effectively used to produce queries that are not included in existing workloads. This allows for the exploration of new query patterns.

Another important advantage of LLM-based generation is its ability to incorporate semantic understanding of schemas. By conditioning on table and column descriptions, LLMs can generate queries that are not only syntactically correct but also semantically meaningful. This capability is particularly useful when dealing with complex schemas, where manual query generation becomes challenging.

Although LLMs can be effectively used to produce queries, they do not have the capability to produce the labels that are associated with the queries. This means that the produced queries have to be executed to produce labels. This is one of the main drawbacks of using LLMs for query generation, as it reintroduces the dependency on database execution.

2.5 Generative AI for Database Components

The base paper [1] attempts to address this data generation bottleneck through a framework that utilizes generative models to automatically generate training data for learned database components. The basic idea is to utilize LLMs to automatically generate realistic SQL queries, thereby reducing reliance on manually collected query logs. This is a major departure from the traditional way learned database systems have been designed. By automating query generation, it is possible to generate large and varied datasets to support a wide range of query types. The framework also outlines how generative models can be incorporated into the learning pipeline to support data generation.

An important contribution of this work is the structured pipeline it proposes, where query generation, validation, execution, and model training are integrated into a unified framework. This pipeline ensures that generated queries are not

only diverse but also executable and meaningful within the given schema. The use of prompt engineering and iterative refinement further improves the quality of generated queries.

One limitation of this framework is that it relies on executing the generated queries to obtain ground truth labels. While query generation is now automated, label generation is computationally intensive and thus limits scalability. This is where a completely synthetic method is proposed, where query and label generation can be performed without relying on database execution. This shift represents a natural evolution of the framework, aiming to remove the remaining bottleneck in the data generation process.

2.6 Active Learning for Data Efficiency

Active learning helps in developing a framework for improving data efficiency, which can be done through the selection of the best data samples for learning. As discussed in [5], not all data samples are equally important in improving the performance of the model. This way, it is possible to attain high accuracy with less data.

In the context of LQE using learned cardinality estimation, active learning helps in improving the efficiency of the process of query labeling. Instead of labeling all the generated queries, the model can be improved through the selection of the best queries in improving the performance of the model. This selection of the best queries can be done using the uncertainty of the model.

In addition to uncertainty-based selection, active learning can also incorporate diversity-based sampling, where queries that represent different regions of the query space are selected. This ensures that the model is exposed to a wide range of patterns while avoiding redundancy in the training data. Such strategies are particularly useful when working with large volumes of synthetically generated data.

The integration of active learning with the process of synthetic data generation helps in improving the efficiency of the learning process. LLMs are capable of generating a wide variety of data samples in the form of queries. Active learning helps in the selection of the best data samples, thus improving the efficiency of the learning process.

2.7 Data Diversity, Personas, and KL Divergence

Diversity in the generated data is a critical factor to be taken into consideration for the training of strong models. The PAARS[7] framework proposes the use of personas for generating diverse and realistic data. Personas help in the simulation of diverse behavior, which enables the generation of diverse data.

In order to test the quality of the generated data, the PAARS framework uses the KL divergence as a similarity metric between the generated data and real-world data. The KL divergence is a measure of the dissimilarity between two probability distributions, which can be used to test the similarity between the generated data and real-world data, especially in the case of SQL queries.

In the context of query generation, KL divergence can be applied to distributions over query features such as the number of joins, predicate types, and selectivity ranges. By comparing these distributions between synthetic and real workloads, it is possible to evaluate whether the generated data accurately reflects real-world patterns. This provides a quantitative basis for improving the quality of synthetic datasets.

2.8 Local Model Hosting for Reproducibility

The practical deployment of LLMs for data generation requires careful consideration of reproducibility, privacy, and cost. In this work, we adopt a local hosting strategy using the Ollama framework, which enables running open-source language models entirely on the researcher's own hardware. This choice offers several important benefits for scientific experimentation. Local hosting eliminates dependency on external API services, ensuring that experiments are not affected by changes in third-party model versions, pricing, or availability. It also preserves data privacy, as database schemas and query workloads never leave the local environment. Furthermore, local inference provides consistent latency characteristics and enables full control over model parameters such as temperature and token limits, supporting reproducible experimental conditions.

2.9 Summary and Motivation

The background provided in this section points to the evolution of learned database systems, starting with conventional estimators, followed by machine learning models, and then moving on to the latest generative models. While the MSCN [4] paper proves the efficiency of learned models, it also points to the disadvantage of data generation. Subsequent papers focus on the need for diversity in the workload[12] and the application of LLMs in query generation [11].

However, the conventional methods are all based on execution-based labeling, which is not scalable. This thesis, with the help of insights provided by generative models, active learning [5], and diversity-aware models, proposes the application of LLMs in generating both the query and the label, thus using the learned model as the final database component.

Chapter 3

Related Work

3.1 Learned Cardinality Estimation

The use of machine learning to replace traditional database components has gained significant attention, particularly in the context of cardinality estimation. A foundational contribution in this area is the Multi-Set Convolutional Network (MSCN) proposed by Kipf et al. [4], which reformulates cardinality estimation as a supervised learning problem. Unlike conventional estimators that rely on simplifying assumptions such as attribute independence, MSCN represents queries as sets of tables, joins, and predicates, allowing the model to capture complex correlations across query components.

A key strength of MSCN lies in its ability to model join-crossing correlations, which are difficult for traditional estimators to handle effectively. Furthermore, the use of bitmap-based sampling enables the model to incorporate information about intermediate query results, improving estimation accuracy for complex queries involving multiple joins and predicates.

Despite these advantages, MSCN fundamentally depends on execution-based labeling. Each training instance requires executing a query on a database to obtain its true cardinality. This introduces substantial computational overhead and limits scalability, particularly when large and diverse workloads are required. Moreover, adapting to new schemas or workloads necessitates additional query execution, further increasing the cost. These limitations highlight a central challenge in learned cardinality estimation: the dependency on labeled data generated through expensive query execution.

3.2 Workload Characteristics and Model Generalization

Beyond model architecture, the effectiveness of learned cardinality estimators is strongly influenced by the characteristics of the training workload. Wang et al. [12] provide a comprehensive analysis demonstrating that learned models are highly sensitive to factors such as predicate types, selectivity, join complexity, and query structure.

A key observation is that models trained on narrow or biased workloads often fail to generalize to unseen queries. For example, a model trained primarily on equality predicates may perform poorly when evaluated on range predicates or more complex join conditions. Similarly, variations in selectivity can significantly impact estimation accuracy, as models struggle to extrapolate beyond observed distributions.

The issue of workload shift further complicates deployment. In real-world sys-

tems, query distributions evolve over time, leading to discrepancies between training and inference workloads. Under such conditions, learned models can exhibit significant performance degradation. These findings emphasize the importance of constructing training datasets that are both diverse and representative of real-world query patterns.

Recent work on robust cardinality estimation under changing workloads [8] further highlights the importance of adaptability, reinforcing the need for data generation approaches that can capture a wide range of query characteristics.

3.3 LLM-based SQL Workload Generation

The emergence of large language models (LLMs) has introduced new possibilities for SQL workload generation. The SQLStorm framework [11] demonstrates that LLMs can generate large-scale, diverse, and realistic SQL queries using schema-aware prompting techniques.

Unlike traditional template-based approaches, which are inherently limited in diversity, LLM-based methods can generate queries with varying levels of complexity, including different join structures, predicate combinations, and nested queries. This flexibility enables the generation of workloads that better reflect real-world usage patterns.

Additional work in text-to-SQL and SQL understanding, such as CHASE-SQL [9] and evaluation studies by Rahaman et al. [10], further demonstrate the capability of LLMs to understand and generate structured queries. Similarly, TAPEX [6] explores pre-training techniques for table reasoning, highlighting the broader applicability of neural models in structured data tasks.

However, existing LLM-based generation approaches primarily focus on query generation rather than labeled data creation. Generated queries still require execution to obtain ground truth cardinalities, thereby retaining the dependency on database systems and limiting scalability for training purposes.

3.4 Generative AI for Training Database Components

Recent work on integrating generative AI into database systems has sought to address the limitations of manual and execution-based data generation. Davitkova and Michel [1] propose a framework that leverages generative models to automate the training of learned database components.

This framework introduces a structured pipeline that integrates query generation, validation, execution, and model training. By automating query generation, it significantly reduces reliance on manually collected query logs and enables the creation of diverse training datasets. Techniques such as prompt engineering and iterative refinement are used to improve the quality and realism of generated queries.

Despite these advancements, the framework still depends on executing queries to obtain ground truth labels. While generation is automated, labeling remains

computationally expensive, limiting scalability. This highlights a critical gap: existing approaches have automated query generation but have not eliminated the dependency on execution-based labeling.

3.5 AL for Efficient Data Utilization

Active learning provides a complementary approach to improving data efficiency in supervised learning systems. Recent advances in deep active learning [5] show that selecting informative samples can significantly reduce the amount of labeled data required for training.

In the context of learned cardinality estimation, active learning can be used to minimize the number of queries that need to be executed for labeling. By prioritizing queries where the model is uncertain or performs poorly, it is possible to achieve high accuracy with fewer labeled samples.

In addition to uncertainty-based strategies, diversity-based sampling plays an important role in ensuring that selected queries cover a wide range of patterns. This is particularly relevant when working with synthetically generated data, where redundancy may otherwise reduce training effectiveness. Combining uncertainty and diversity criteria allows for the construction of efficient and representative training datasets.

Although active learning has been extensively studied in machine learning, its application to learned database systems remains relatively limited. Integrating active learning with LLM-based data generation presents a promising direction for improving both scalability and efficiency.

3.6 Data Diversity and Distribution Alignment

Ensuring that synthetic data reflects real-world distributions is a critical challenge. The PAARS framework [7] introduces persona-based generation to simulate diverse user behaviors, enabling the creation of varied and realistic data.

A key contribution of this work is the use of KL divergence to measure the similarity between generated and real data distributions. This provides a quantitative method for evaluating whether synthetic data accurately captures real-world characteristics. In the context of SQL workloads, such metrics can be applied to distributions over query features, including join counts, predicate types, and selectivity.

Distribution alignment is essential for training robust models. If synthetic data does not reflect real-world usage patterns, learned models may fail to generalize effectively. Incorporating distributional metrics into the generation process helps ensure that synthetic datasets are both diverse and representative.

3.7 Positioning of the Proposed Approach

The existing literature demonstrates significant progress in learned database systems and LLM-based data generation. Models such as MSCN [4] highlight

the potential of learned approaches for cardinality estimation, while studies on workload characteristics [12] emphasize the importance of diverse training data. LLM-based frameworks such as SQLStorm [11] and generative pipelines [1] demonstrate the feasibility of large-scale query generation.

However, a common limitation across these approaches is the continued reliance on execution-based labeling. While query generation has been largely automated, obtaining ground truth labels still requires executing queries on a database, introducing significant computational overhead.

The approach proposed in this thesis addresses this limitation by leveraging LLMs to generate both queries and their corresponding labels, thereby eliminating the need for database execution during training. Furthermore, by incorporating active learning [5] and diversity-aware generation strategies [7], the proposed method aims to improve both data efficiency and representativeness.

In this framework, the LLM serves as a data generator, while the trained supervised model acts as the final database component during inference. This separation enables scalable and efficient training pipelines, representing a step toward fully automated learned database systems.

Chapter 4

Approach

This chapter presents the overall approach for creating and benchmarking learned database components using large language models and active learning. We introduce the system architecture at a conceptual level and describe the core ideas that motivate our pipeline design. The detailed algorithmic treatments of the query generation methodology and the training process with active learning are deferred to Chapters 5 and 6, respectively. Experimental results including workload analysis, learning curves, and model evaluation are presented in Chapter 7.

4.1 Benefits of the Proposed Framework

The proposed framework offers several important benefits that distinguish it from prior work on learned cardinality estimation. First, the system is designed to be schema-agnostic: given a schema definition containing table names, column names, data types, and foreign key relationships, the system automatically generates appropriate queries, validates them against the schema, and produces correctly structured training data without any manual configuration. This means that the same pipeline can be applied to entirely different databases without modification, making it a general-purpose tool for training learned estimators.

Second, the framework is built around modular and extensible components. The query generation module can work with any LLM provider or fall back to a deterministic synthetic generator. The active learning module supports pluggable acquisition functions, and the model architecture can be swapped without affecting the rest of the pipeline. This modularity enables researchers to isolate and evaluate individual contributions independently.

Third, the system provides production-grade robustness through timeout handling for long-running database queries, automatic retry logic with exponential backoff for transient errors, and graceful degradation when queries fail after maximum retries. These mechanisms ensure that the pipeline can process thousands of queries without manual intervention, even when operating on complex schemas with expensive join queries.

Finally, the framework supports full reproducibility through configurable random seeds, complete experiment configuration logging, CSV-based results export, and timestamped output directories. These features are essential for scientific rigor and enable meaningful comparisons across different experimental configurations.

4.2 System Architecture

The proposed system consists of an end-to-end pipeline that integrates six major stages: query generation, query validation and format conversion, database labeling, feature extraction and bitmap generation, active learning with model training, and evaluation with visualization. Figure 4.1 illustrates the high-level architecture.

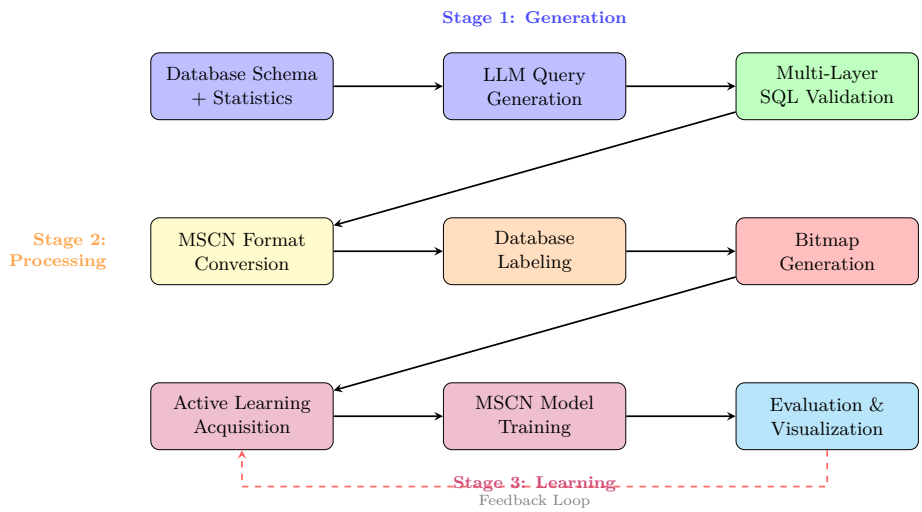

Figure 4.1: High-level system architecture showing the end-to-end pipeline from schema ingestion through query generation, validation, labeling, and active learning to final model evaluation. The dashed feedback loop indicates iterative refinement through multiple active learning rounds.

4.3 Pipeline Overview

The pipeline begins with query generation, where a large pool of unlabeled SQL queries is produced either by prompting a large language model with the database schema or by using a deterministic synthetic generator. Unlike traditional template-based generators that rely on predefined patterns, LLMs capture complex linguistic and structural patterns present in real-world workloads, producing queries with greater diversity in join topologies, predicate combinations, and selectivity profiles. The details of the prompt engineering strategy, diversity mechanisms, and the multi-layer validation pipeline that filters invalid LLM output are presented in Chapter 5.

Once a pool of valid queries has been assembled, the pipeline proceeds to database labeling, where ground truth cardinalities are obtained by executing each query against the target database. This is the most computationally expensive stage of the pipeline, and it is precisely this cost that motivates the use of active learning. Rather than labeling the entire query pool, only a strategically chosen subset is executed, dramatically reducing the total computation required.

Before queries can be fed to the neural network, they must be transformed into fixed-size tensor representations. The feature extraction stage encodes each query as three sets of vectors representing tables, predicates, and joins. A key element of this encoding is the bitmap feature, which captures the selectivity of each query's filter predicates on a pre-sampled subset of the underlying data. These bitmaps provide the model with data-aware signals that go beyond purely structural features, enabling it to learn correlations between query predicates and actual data distributions.

The active learning loop forms the core optimization mechanism of the pipeline. It operates iteratively: an initial batch of queries is randomly selected and labeled to bootstrap the model, after which the model is trained and then used to score all remaining unlabeled queries according to an acquisition function. The highest-scoring queries, those where the model is most uncertain or where the greatest learning potential exists, are selected for labeling and added to the training set. This cycle repeats for a configurable number of rounds, progressively building a training set that is both compact and highly informative. The framework supports multiple acquisition strategies including random sampling, uncertainty sampling, and Monte Carlo dropout, each offering different trade-offs between computational cost and sample efficiency. The complete training methodology, MSCN architecture, and active learning strategies are detailed in Chapter 6.

The final stage produces comprehensive evaluation metrics and visualizations including Q-error distributions, learning curves, predicted-versus-actual scatter plots, and workload analysis dashboards. These outputs enable detailed assessment of both the generated workload quality and the trained model's estimation accuracy.

4.4 Summary

This chapter introduced the overarching approach for creating and benchmarking learned database components using LLMs and active learning. We described the benefits of the proposed framework, presented the six-stage pipeline architecture, and outlined the flow of data from schema ingestion through query generation and validation to active learning and model evaluation. The following chapters present the detailed algorithmic treatments: Chapter 5 covers the LLM-based query generation approach, and Chapter 6 details the training methodology including feature extraction, active learning, and model architecture.

Chapter 5

Query Generation Approach

This chapter presents the detailed approach for generating diverse, realistic SQL query workloads using large language models. Query generation is the foundational stage of the pipeline, as the quality and diversity of generated queries directly impacts the effectiveness of downstream model training. We describe the prompt engineering strategy, the multi-layer validation pipeline, the format conversion process, and the synthetic fallback generator.

5.1 LLM-Based Query Generation

Traditional approaches to generating training workloads for learned database components fall into two broad categories: manual collection from query logs and synthetic generation using template-based or random sampling methods. Query logs require access to production systems, raise privacy concerns, and may not cover the full spectrum of query patterns needed for robust model training. Template-based generators, on the other hand, produce queries that are syntactically valid but often lack the structural variety and realistic predicate distributions found in real workloads. Random sampling can generate uniformly distributed queries but misses the complex correlations and patterns that characterize real-world usage.

Large language models offer a compelling alternative because they have been trained on vast corpora of natural language and code, including millions of SQL queries. This training enables them to generate queries that exhibit realistic structural patterns, meaningful predicate combinations, and diverse join topologies without explicit programming of these patterns. Our framework uses the Ollama platform to host and serve LLMs locally, ensuring data privacy and eliminating dependency on external API services. The default model is Llama 3.2, selected for its strong code generation capabilities and reasonable computational requirements. The system interacts with Ollama through its REST API, enabling seamless switching between different model sizes and families without code changes.

5.2 Prompt Design

The quality of LLM-generated queries critically depends on the prompt design. We employ a carefully structured prompt template that provides the model with sufficient context to generate valid, diverse queries while constraining output to the supported query fragment. The prompt consists of five hierarchically organized components, as illustrated in Figure 5.1: a role assignment that establishes the LLM as a SQL workload generator, the complete database schema context including DDL statements with table names, columns, data types, and foreign

key relationships, optional column statistics providing min/max values for numeric columns to enable realistic predicate generation, a set of strict generation rules, and an output format specification.

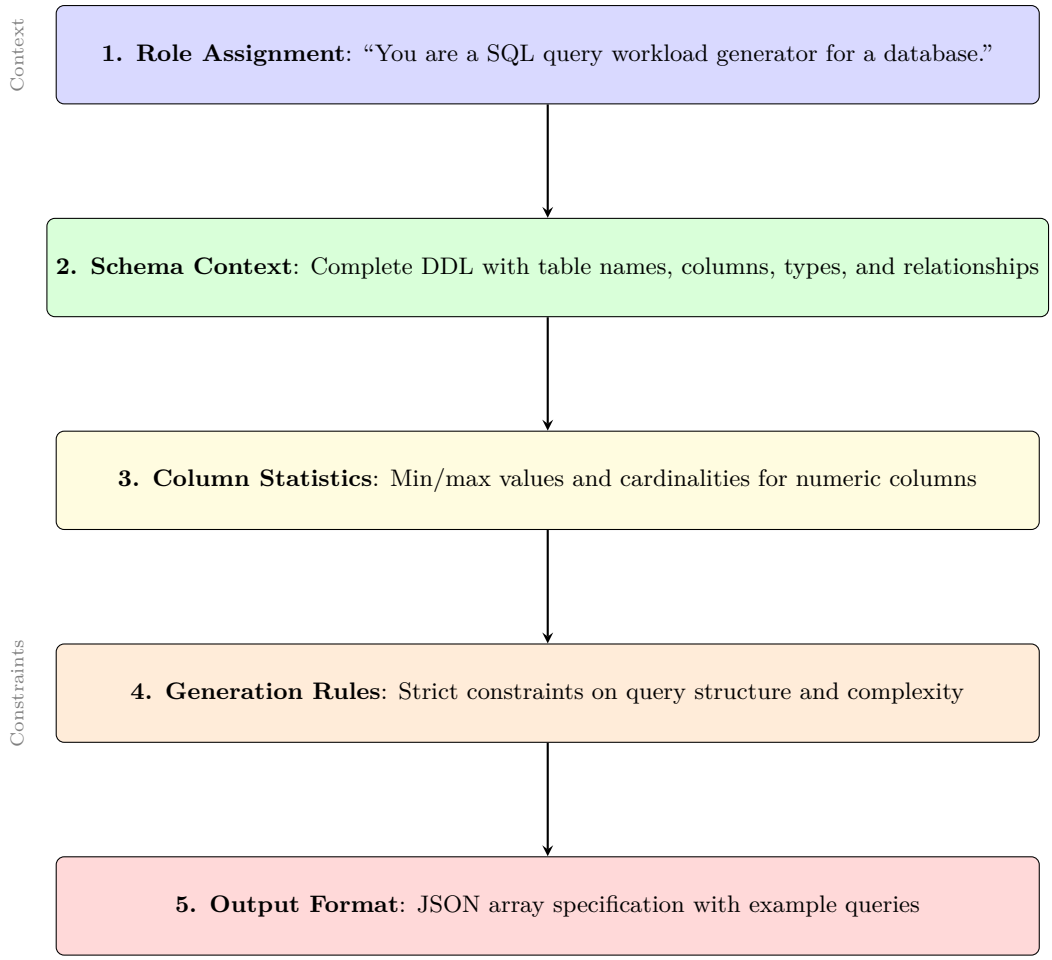

Figure 5.1: Structure of the LLM prompt template, consisting of five hierarchically organized components that guide the model from context understanding to constrained output generation.

The generation rules are critical for ensuring that the produced queries are compatible with the MSCN encoding. The prompt enforces that all queries use the `SELECT COUNT(*)` format exclusively, involve one to three tables from the provided schema with proper aliases, include join conditions using defined foreign key relationships, and use only simple comparison operators (`=`, `<`, `>`) with numeric values connected by `AND`. Constructs such as `OR`, `IN`, `LIKE`, `BETWEEN`, subqueries, and aggregations are explicitly forbidden. These constraints are necessary because the MSCN encoding assumes a specific query structure where predicates are AND-connected simple comparisons over numeric columns; queries violating these constraints would either fail during encoding or produce incorrect feature representations.

The prompt instructs the LLM to return queries as a JSON array of objects, each containing a single `"sql"` field. This structured output format enables reliable parsing and reduces the need for complex text extraction. A fallback regex-based extraction is used when the LLM includes additional text around the JSON.

5.3 Diversity and Batch Generation

To maximize workload diversity across generation batches, we employ a rotating set of diversity hints. Each batch receives a different textual hint that steers the LLM toward generating queries with specific characteristics, ranging from single-table queries with various filters to complex three-table joins with multiple predicates, and from point lookups to queries with extreme selectivities. The hint index is computed as $(\lfloor \text{existing_count}/\text{batch_size} \rfloor) \bmod 5$, ensuring systematic rotation through all diversity profiles as generation progresses. This approach avoids the mode collapse problem where the LLM repeatedly generates structurally similar queries.

Generating all queries in a single LLM call would exceed token limits and reduce diversity. Instead, the system generates queries in configurable batches, with the total number of batches determined by $\lceil \text{total_queries}/\text{batch_size} \rceil$. Each batch receives a fresh prompt with the current diversity hint, ensuring variety across the full workload. LLM generation is inherently stochastic and may fail due to network errors, malformed JSON output, or hallucination. Each batch is therefore retried up to three times with increasing delays between attempts, achieving near-complete batch success rates in practice. Algorithm 1 presents the batch generation procedure.

Algorithm 1 LLM-Based Query Generation

```
Determine how many batches are needed to reach the target number of queries
for each batch do
    Choose a diversity hint based on how many queries have been generated so far
    Construct a prompt containing the database schema, column statistics, and the chosen hint
    for up to 3 attempts do
        Send the prompt to the language model and receive a response
        Try to extract valid SQL queries from the response
        if at least one valid query was found then
            Add the extracted queries to the collection and move to the next batch
        end if
    end for
end for
return the collected queries, trimmed to the target count
```

5.4 Multi-Layer Validation Pipeline

Raw LLM output must undergo rigorous validation before entering the training pipeline. Despite careful prompt engineering, LLMs can produce queries that are syntactically invalid, reference non-existent schema elements, or use unsupported SQL constructs. Our validation pipeline applies four sequential filtering stages, each progressively more expensive but more thorough, as illustrated in Figure 5.2.

The first layer is a fast, regex-based filter that rejects queries containing forbidden SQL patterns such as `OR`, `IN`, `LIKE`, `BETWEEN`, subqueries, and aggregations. This layer catches the most common LLM errors at minimal computational cost. The second layer parses each surviving query to extract its FROM and WHERE clauses, handling both comma-separated join syntax and explicit `JOIN...ON` syntax and producing a normalized representation of tables, join conditions, and filter predicates.

The third layer validates the parsed query against the database schema, verifying that all referenced table names, column names, and aliases actually exist. Join conditions are checked to confirm that they reference columns participating in defined foreign key relationships. The fourth and final layer converts valid queries from their parsed representation into the MSCN-compatible structured format, decomposing each query into three sets: tables (with aliases), joins (as equality conditions), and predicates (as column-operator-value tuples). This conversion includes operator standardization, value normalization, case normalization, and alias resolution to ensure consistent encoding downstream.

5.5 Synthetic Query Generator

For testing and environments where an LLM is unavailable, the framework includes a deterministic synthetic query generator. This generator produces structurally valid queries using parameterized random sampling based on hardcoded table definitions from the target schema. It uses weighted random selection for table count, biasing toward two-table queries to match typical workload distributions, and all multi-table queries include the central fact table to reflect the star-schema topology of the benchmark database.

Table 5.1 compares the characteristics of queries produced by the two generation approaches.

Table 5.1: Comparison of LLM-based and synthetic query generation approaches.

Property	**LLM-Based**	**Synthetic**
Schema adaptability	Automatic	Requires manual updates
Predicate diversity	High (learned patterns)	Medium (uniform random)
Join topology variety	High	Limited to star schema
Value distribution	Realistic (stat-aware)	Uniform within range
Validation pass rate	∼95%	100% (by construction)
Reproducibility	Non-deterministic	Fully deterministic

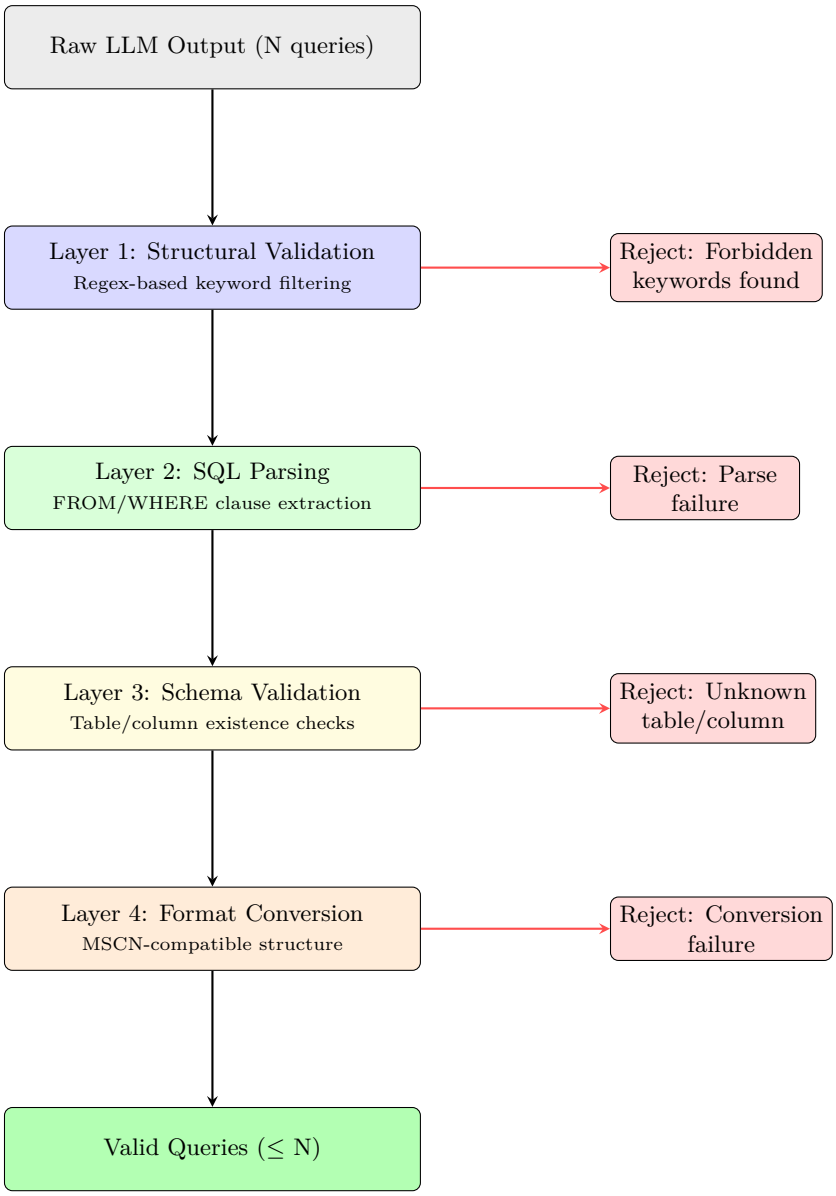

Figure 5.2: Multi-layer validation pipeline for LLM-generated queries. Each layer progressively filters invalid queries, with rejected queries logged for analysis.

The LLM-based generator produces higher-quality workloads with greater diversity, while the synthetic generator provides a deterministic, zero-dependency fallback. In practice, the LLM-based generator is recommended for final experiments, while the synthetic generator serves well for rapid development and testing.

5.6 Summary

This chapter presented the detailed approach for generating SQL query workloads using large language models. The key contributions include a structured prompt engineering strategy that achieves high validation pass rates, a four-layer validation pipeline that ensures only well-formed and schema-compatible queries enter the training pipeline, a diversity hint rotation mechanism that prevents mode collapse in generated workloads, and a deterministic synthetic generator for testing environments. The next chapter describes how these generated queries are labeled, encoded, and used to train the MSCN cardinality estimator through active learning.

Chapter 6

Training Approach

This chapter presents the training methodology for the learned cardinality estimator, encompassing database labeling, feature extraction, the Multi-Set Convolutional Network architecture, active learning strategies, and the evaluation framework. Together, these components transform generated queries into a trained model capable of accurate cardinality prediction.

6.1 Database Labeling

Ground truth cardinalities are obtained by executing reconstructed `SELECT COUNT(*)` queries against the target database. Each query's tables, joins, and predicates are assembled into a complete SQL statement and submitted to the database engine. Since generated workloads contain queries of widely varying complexity, some involving expensive multi-table joins over tens of millions of rows, the labeling pipeline enforces per-query timeouts and implements retry logic to ensure that no single query can block the pipeline. Queries that fail after all retries are assigned a fallback cardinality of one, satisfying the log-normalization requirement of the model while allowing the pipeline to continue. All cardinalities are clamped to a minimum of one because the training loss operates in log-space, where $\log(0)$ is undefined.

The labeling stage is the most computationally expensive part of the pipeline. A single query on the IMDB dataset can take several seconds to execute, and labeling thousands of queries may require hours of database time. This cost is the primary motivation for active learning: by intelligently selecting which queries to label, the framework achieves strong model performance while executing only a fraction of the total query pool.

6.2 Feature Extraction and Bitmap Generation

The MSCN architecture requires fixed-size tensor inputs, but queries have variable numbers of tables, predicates, and joins. The feature extraction stage addresses this through set-based encoding with zero-padding and masking.

Before encoding any queries, the system builds encoding vocabularies from the complete query pool. A table vocabulary provides one-hot encodings over all unique table entries, a column vocabulary covers all unique predicate columns, an operator vocabulary covers the observed comparison operators, and a join vocabulary covers all unique join conditions. These vocabularies are sorted alphabetically to ensure deterministic encoding across runs.

Each query is then encoded into three sets of feature vectors. For each table t_i in the query, the feature vector concatenates the table's one-hot encoding

with a bitmap vector that captures the selectivity of the query's predicates on a pre-sampled set of rows:

$$\mathbf{s}_i = [\text{onehot}(t_i) \| \text{bitmap}(t_i)] \in \mathbb{R}^{|T|+n_{\text{samples}}} \tag{6.1}$$

For each predicate (c_j, op_j, v_j), the feature vector concatenates a column one-hot vector, an operator one-hot vector, and the normalized predicate value:

$$\mathbf{p}_j = [\text{onehot}(c_j) \| \text{onehot}(op_j) \| \text{norm}(v_j)] \in \mathbb{R}^{|C|+|O|+1} \tag{6.2}$$

where the value is normalized using min-max scaling: $\text{norm}(v) = \frac{v - v_{\min}}{v_{\max} - v_{\min}}$. Each join condition j_k is simply represented as its one-hot encoding $\mathbf{j}_k = \text{onehot}(j_k) \in \mathbb{R}^{|J|}$.

Since queries have variable numbers of components, each set is padded to the maximum observed size and binary masks are generated to indicate which positions contain real data versus padding:

$$\text{mask}(i) = \begin{cases} 1 & \text{if position } i \text{ contains real data} \\ 0 & \text{if position } i \text{ is padding} \end{cases} \tag{6.3}$$

The masks are used during the mean pooling operation to compute averages only over non-padded positions, preventing dilution of the learned representations.

The bitmap features deserve special attention, as they are the mechanism through which the model gains awareness of the underlying data distribution. At pipeline initialization, a fixed set of rows is randomly sampled from each table and materialized in memory. For each query, the system evaluates which of these sampled rows satisfy the query's filter predicates on each table, producing a binary vector where each bit indicates whether the corresponding sampled row passes the filter. This approach offers fixed dimensionality regardless of table size, composability across multi-table queries, and reusability of the same materialized samples for all queries in the workload.

6.3 Multi-Set Convolutional Network

The MSCN architecture, proposed by Kipf et al. [4], treats a SQL query as a composition of three independent sets—tables with bitmaps, predicates, and joins—each processed through set-specific neural modules before being combined for the final prediction. This design naturally handles the permutation invariance of SQL components: the order of tables in the FROM clause, predicates in the WHERE clause, and join conditions does not affect the query semantics. Figure 6.1 illustrates the complete architecture.

Each set module processes its input through a two-layer MLP with ReLU activations and dropout regularization. The first layer projects the input features to a hidden dimension h, and the second layer refines the representation:

$$\mathbf{h}_i^{(1)} = \text{Dropout}\left(\text{ReLU}\left(\mathbf{W}_1 \mathbf{x}_i + \mathbf{b}_1\right), p\right) \tag{6.4}$$

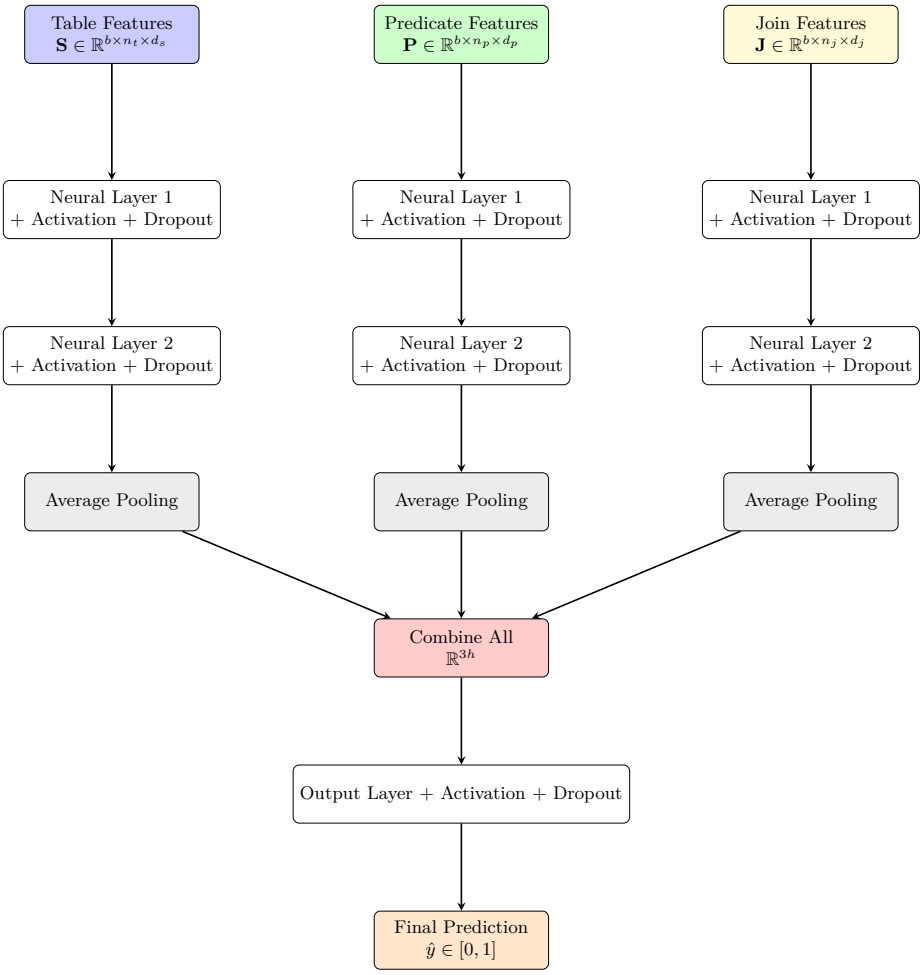

Figure 6.1: Complete MSCN architecture. Each query component (tables, predicates, joins) passes through a dedicated two-layer neural network with dropout regularization, followed by average pooling. The pooled representations are combined and processed through an output layer to produce a normalized cardinality estimate $\hat{y} \in [0, 1]$.

$$\mathbf{h}_i^{(2)} = \text{Dropout}\left(\text{ReLU}\left(\mathbf{W}_2\mathbf{h}_i^{(1)} + \mathbf{b}_2\right), p\right) \tag{6.5}$$

The masked mean pooling operation then aggregates the set elements while respecting the padding mask, computing the average only over positions containing real data:

$$\mathbf{h}_{\text{pool}} = \frac{\sum_{i=1}^{n} m_i \cdot \mathbf{h}_i^{(2)}}{\sum_{i=1}^{n} m_i} \tag{6.6}$$

The concatenated pooled representations from all three modules pass through an output network that produces the final normalized cardinality estimate:

$$\hat{y} = \sigma\left(\mathbf{W}_{\text{out2}} \cdot \text{Dropout}\left(\text{ReLU}\left(\mathbf{W}_{\text{out1}}[\mathbf{h}_{\text{sample}} \| \mathbf{h}_{\text{pred}} \| \mathbf{h}_{\text{join}}] + \mathbf{b}_{\text{out1}}\right)\right) + \mathbf{b}_{\text{out2}}\right) \tag{6.7}$$

The sigmoid activation constrains the output to $[0, 1]$, representing a normalized cardinality estimate.

6.4 Label Normalization and Loss Function

True cardinalities span many orders of magnitude, from single-digit counts to tens of millions. Passing raw integer values to the loss function would cause catastrophic gradient imbalances, as the model would be dominated by large-cardinality queries. We normalize labels using log-scale min-max normalization:

$$y_{\text{norm}} = \frac{\log(y) - \log_{\text{min}}}{\log_{\text{max}} - \log_{\text{min}}} \tag{6.8}$$

where $\log_{\text{min}}$ and $\log_{\text{max}}$ are the minimum and maximum log-cardinalities in the training set. At inference time, predictions are unnormalized by inverting this transformation:

$$\hat{y}_{\text{actual}} = \exp\left(\hat{y}_{\text{norm}} \cdot (\log_{\text{max}} - \log_{\text{min}}) + \log_{\text{min}}\right) \tag{6.9}$$

We use Q-error as both the training loss and the evaluation metric. Q-error is the standard measure for cardinality estimation quality, defined as:

$$Q(p, a) = \max\left(\frac{p}{a}, \frac{a}{p}\right) \tag{6.10}$$

where p is the predicted cardinality and a is the actual cardinality. Q-error possesses two desirable properties: it is symmetric, meaning that overestimation and underestimation by the same factor yield the same error, and it is scale-invariant, treating a factor-of-two error identically whether the true cardinality is one hundred or one million. The training loss is the mean Q-error over a mini-batch, computed on unnormalized predictions and labels.

6.5 Active Learning Framework

Active learning addresses the central question of this work: given a fixed labeling budget, which queries should be labeled to maximize model performance? Formally, let $\mathcal{U} = \{q_1, q_2, \ldots, q_N\}$ be the unlabeled query pool, let $\mathcal{L} \subset \mathcal{U}$ be the current labeled set with $|\mathcal{L}| \ll |\mathcal{U}|$, and let f_θ be the MSCN model trained on $\mathcal{L}$. The goal is to select a batch $\mathcal{B} \subset \mathcal{U} \setminus \mathcal{L}$ of size k that maximizes the improvement in f_θ's performance on a held-out validation set $\mathcal{V}$. The selection is guided by an acquisition function $\alpha : \mathcal{U} \setminus \mathcal{L} \to \mathbb{R}$ that scores each unlabeled query by its estimated informativeness.

The framework supports three acquisition strategies. Random sampling serves as the baseline, selecting queries uniformly at random from the unlabeled pool with $\alpha_{\text{random}}(q) = \text{Uniform}[0, 1]$. Despite its simplicity, random sampling provides reasonable performance due to the inherent diversity of the query pool.

Uncertainty sampling selects queries where the model is most uncertain about its prediction. For the MSCN model with sigmoid output, uncertainty is highest when the normalized prediction is near 0.5, yielding $\alpha_{\text{uncertainty}}(q) = -|f_\theta(q) - 0.5|$. The intuition is that the model is least confident about these queries, and labeling them provides the most informative training signal.

Monte Carlo Dropout [2] provides a Bayesian approximation of model uncertainty by performing T stochastic forward passes with dropout enabled at inference time. The variance across these predictions serves as the uncertainty estimate:

$$\alpha_{\text{MC}}(q) = \text{Var}\left(\{\hat{y}^{(t)}\}_{t=1}^{T}\right) = \frac{1}{T}\sum_{t=1}^{T}\left(\hat{y}^{(t)} - \bar{y}\right)^2 \tag{6.11}$$

where $\bar{y} = \frac{1}{T}\sum_{t=1}^{T}\hat{y}^{(t)}$ is the mean prediction. Queries with the highest variance are selected, as high variance indicates that the model's predictions are sensitive to which neurons are dropped, implying that the model has not yet learned a stable representation for these queries.

Algorithm 2 presents the complete active learning procedure.

The total number of queries labeled across the active learning process is $|\mathcal{L}_{\text{total}}| = \lfloor 0.2 \times |\mathcal{P}| \rfloor + R \times k_{\text{acquire}} + |\mathcal{V}|$, where $|\mathcal{P}|$ is the pool size, R is the number of rounds, k_{acquire} is the per-round acquisition size, and $|\mathcal{V}|$ is the validation set size. The initial labeled set comprises 20% of the pool, providing sufficient data for the model to learn a reasonable baseline before active acquisition begins.

6.6 Summary

This chapter presented the complete training approach for the learned cardinality estimator, covering the database labeling process, the set-based query encoding with bitmap features, the MSCN neural network architecture with masked mean pooling, the Q-error loss function, and the active learning framework with three acquisition strategies. The active learning strategies aim to

Algorithm 2 Active Learning for Cardinality Estimation

```
Set aside 10% of all queries as a fixed validation set
From the remaining queries, randomly pick 20% as the initial training set
Obtain true cardinalities for the initial training set by running them on the database
Create a new MSCN model
for each active learning round do
    Train the model on all currently labeled queries
    Measure the model's accuracy on the validation set
    Use the chosen strategy to score each unlabeled query by how informative it would be
    Select the top-scoring queries and obtain their true cardinalities from the database
    Add the newly labeled queries to the training set
end for
return the trained model and accuracy metrics
```

achieve significant improvements in sample efficiency compared to random sampling, reducing the number of expensive database executions needed to train an accurate model. The following chapter presents the experimental evaluation of the complete pipeline.

Chapter 7

Experimental Evaluation

This chapter presents the experimental evaluation of the complete pipeline. We first describe the experimental setup, including the dataset, system environment, and hyperparameter configuration. We then analyze the quality of the generated query workload and evaluate the active learning process together with the accuracy of the trained cardinality estimator. All figures in this chapter are generated automatically by the evaluation module of the pipeline, providing an accurate and reproducible representation of the system's behavior on real data.

7.1 Experimental Setup

All experiments are conducted on the Internet Movie Database (IMDB), a widely used benchmark in learned cardinality estimation research [4]. The IMDB dataset consists of four core tables: `title_basics` containing metadata for approximately 10 million titles including title type, production year, runtime, and genre information; `name_basics` containing metadata for approximately 12 million people associated with titles; `title_principals` serving as a junction table linking titles to the principal cast and crew with approximately 56 million records; and `title_ratings` containing user ratings and vote counts for approximately 1.3 million titles. The schema defines foreign key relationships between tables through the `tconst` and `nconst` identifiers, enabling meaningful join queries. Figure 7.1 illustrates the schema relationships.

Table 7.1 summarizes the configuration parameters used across all experiments. These values were selected based on preliminary experiments and align with recommended settings from the MSCN literature. The pipeline is implemented in Python using PyTorch for the neural network training. PostgreSQL serves as the database backend, and Ollama provides local LLM hosting. All experiments are executed on a standard workstation.

7.2 Query Generation Analysis

The quality of the generated query workload is a critical factor in training an effective estimator. Figure 7.2 shows the combined distribution of tables, joins, and predicates across all generated queries. The distributions reveal a balanced mix of single-table, two-table, and three-table queries, ensuring that the model is trained on queries of varying structural complexity. The predicate count distribution demonstrates that most queries include between one and three filter conditions, reflecting realistic workload patterns.

Figure 7.3 shows the query validation success rate across the four validation layers. The high pass rate demonstrates that the prompt engineering strategy effectively guides the LLM to produce well-formed, schema-compatible queries, with only a small fraction rejected at each validation stage.

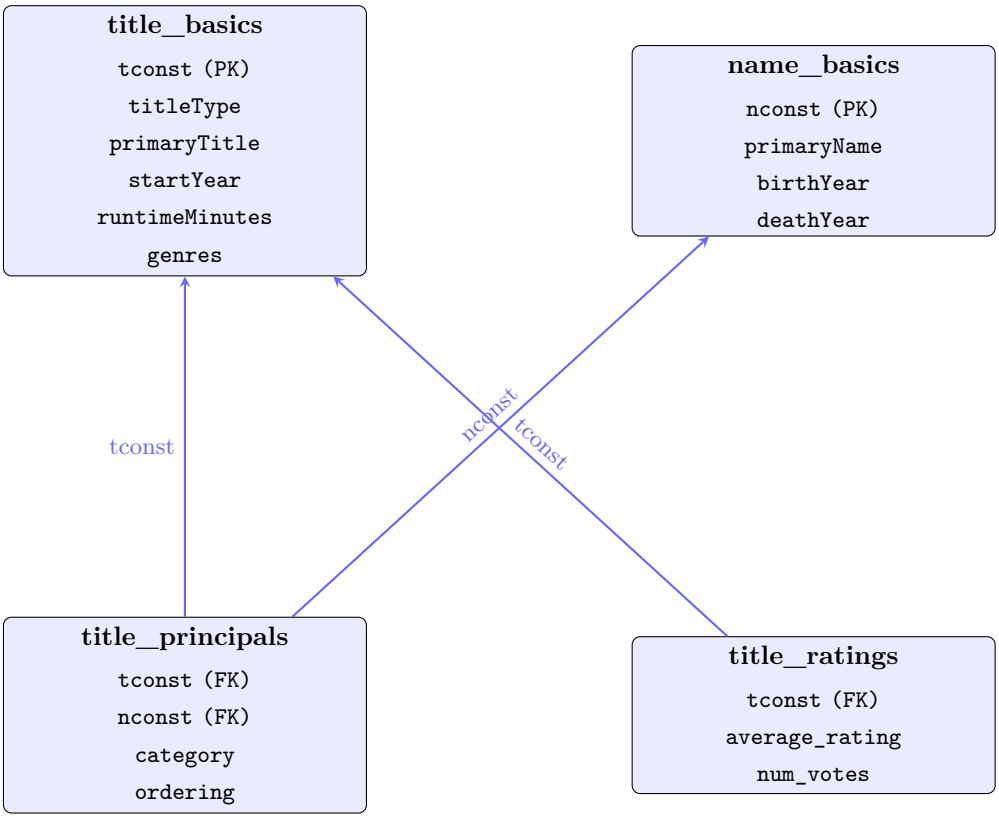

Figure 7.1: Entity-relationship diagram of the IMDB database schema. The `title_principals` table acts as a central junction connecting titles to people, while `title_ratings` provides aggregate user feedback per title. These relationships define the valid join paths for generated queries.

Table 7.1: Experimental configuration parameters.

Category	**Parameter**	**Value**
Query Generation	Total queries	5000
	LLM batch size	20
	LLM model	Llama 3.2
Database	Materialized samples	1000
	Max retries	2
Active Learning	Strategy	random / uncertainty / mc_dropout
	Rounds	5
	Acquisition batch size	200
	MC Dropout passes (T)	25
MSCN Training	Hidden units	256
	Learning rate	10^{-4}
	Epochs per round	10
	Training batch size	1024

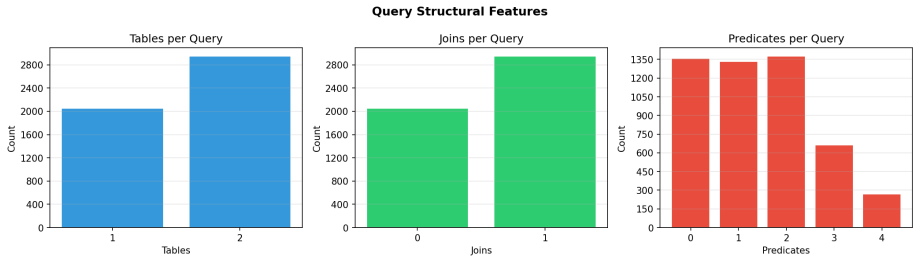

Figure 7.2: Structural feature distributions across the generated workload. The balanced coverage of different table counts, join complexities, and predicate counts confirms that the diversity hint mechanism prevents the LLM from converging on a narrow subset of query patterns.

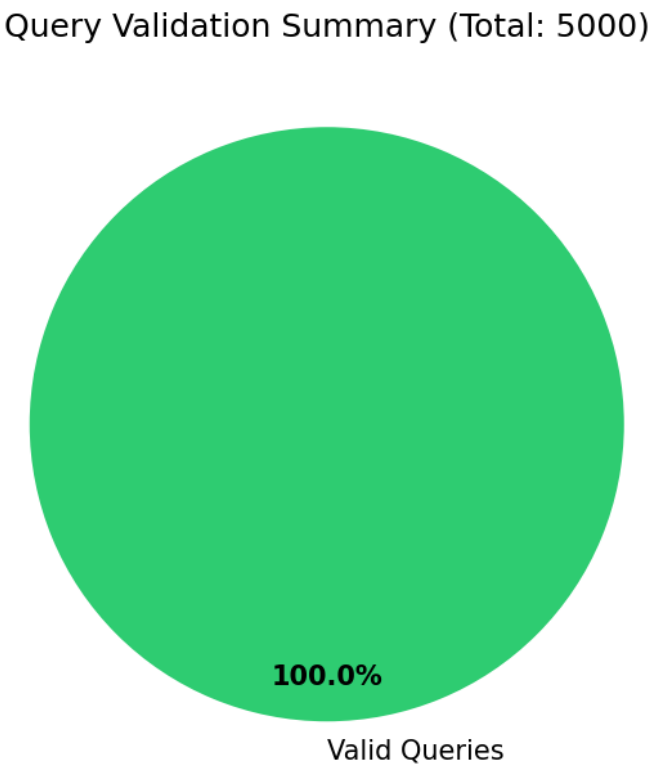

Figure 7.3: Query validation success rates across the four filtering layers. The high pass rate at each stage demonstrates the effectiveness of the prompt engineering constraints in guiding the LLM toward valid query structures.

Figure ?? provides a closer look at the individual distributions of tables and predicates per query, while Figure 7.6 shows the join condition distribution separately.

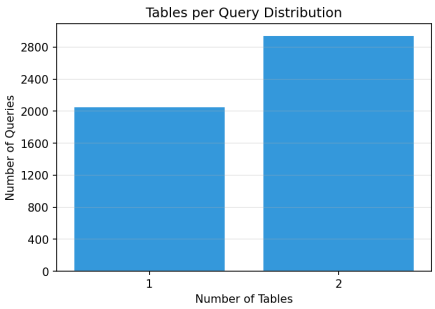

Figure 7.4: Distribution of table count per query. The weighted mix of single-table, two-table, and three-table queries ensures coverage of both simple selections and complex multi-way joins.

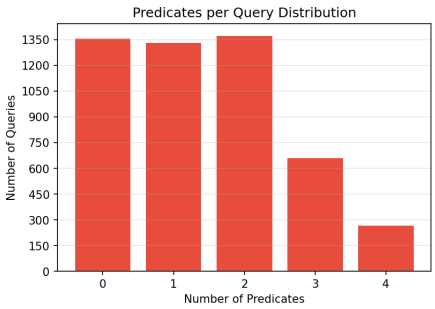

Figure 7.5: Distribution of predicate count per query. The majority of queries contain one to three predicates, reflecting a realistic range of filter complexities.

Figure 7.7 shows the distribution of true cardinalities for the labeled queries on a logarithmic scale. The distribution spans from single-digit cardinalities to tens of millions of rows, which is essential for training a model that generalizes across the full selectivity spectrum. Without this range, the model would be unable to distinguish between highly selective filters and broad scans.

7.3 Active Learning and Model Training Results

This section presents the results from the active learning loop, examining how the model's accuracy evolves across rounds and how effectively it learns to predict query cardinalities.

Figure 7.8 shows the active learning curve, plotting the validation median Q-error against the number of labeled training queries. The decreasing trend confirms that the model benefits from each active learning round, achieving progressively lower estimation errors as the training set grows with informatively selected queries.

Figure 7.9 shows how the training loss evolves across active learning rounds. Each subsequent round starts from a lower loss baseline, confirming that the model retains useful representations from earlier rounds while benefiting from the newly acquired training data.

Figure 7.10 presents a scatter plot of predicted versus actual cardinalities on a log-log scale. Points near the diagonal line $y = x$ indicate accurate predictions, while deviations reveal systematic estimation biases. The tight clustering around the diagonal across several orders of magnitude demonstrates that the trained model achieves good calibration for both small and large cardinalities.

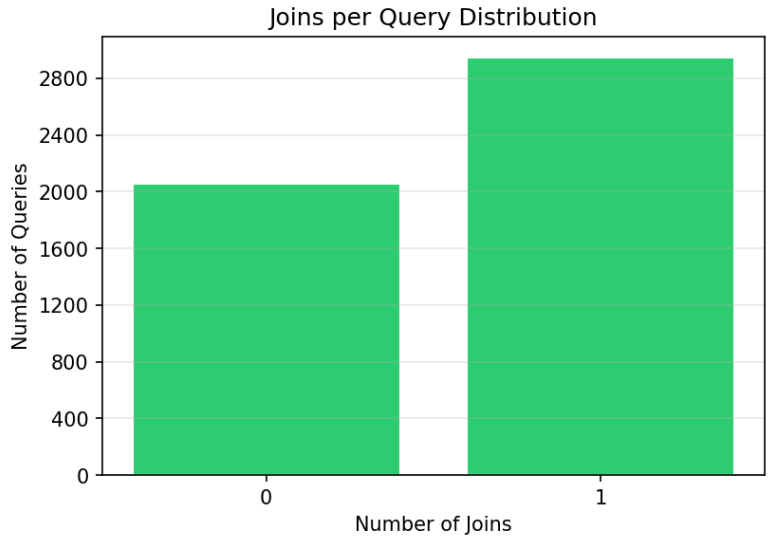

Figure 7.6: Distribution of join conditions per query. Queries with zero joins correspond to single-table queries, while one and two join conditions correspond to two-table and three-table queries, respectively.

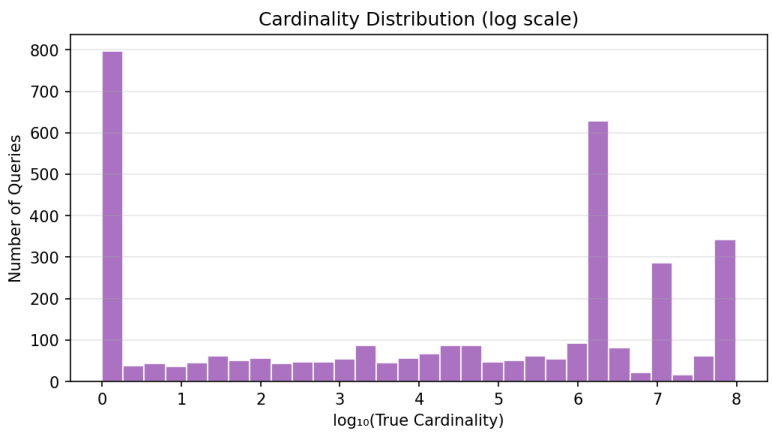

Figure 7.7: Distribution of true cardinalities ($\log_{10}$ scale). The wide range spanning several orders of magnitude forces the model to learn accurate estimates for both highly selective queries returning few rows and broad queries spanning millions of tuples.

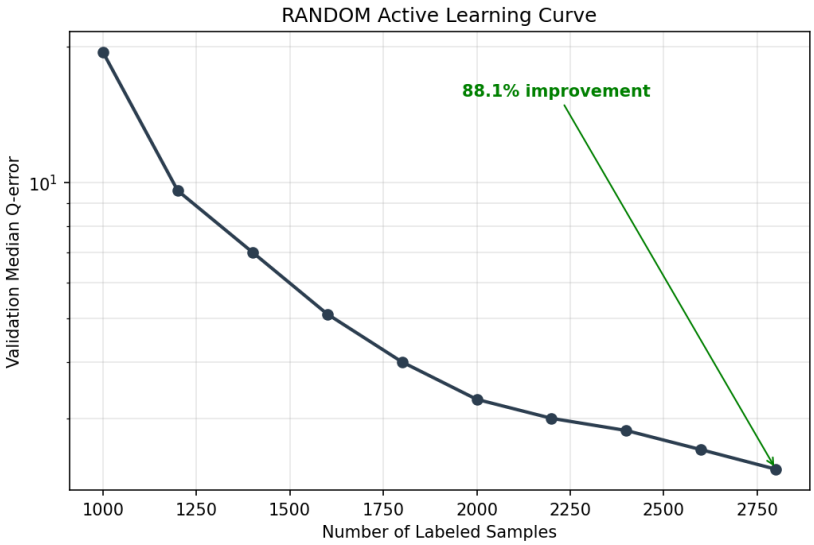

Figure 7.8: Active learning curve showing validation median Q-error versus the number of labeled queries. The steep initial decline demonstrates rapid learning from the first few rounds, while the flattening tail indicates diminishing returns as the model approaches its capacity on the given data.

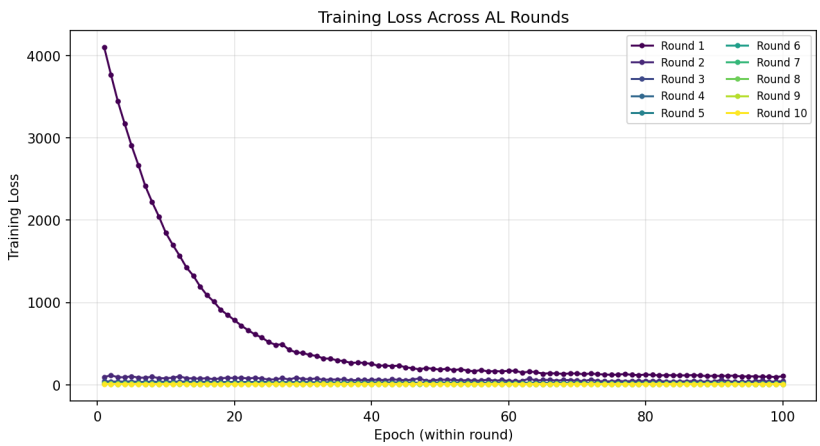

Figure 7.9: Training loss curves across active learning rounds. The consistently decreasing trajectory demonstrates stable convergence, with each round starting from a better baseline than the previous one due to the expanded and more informative training set.

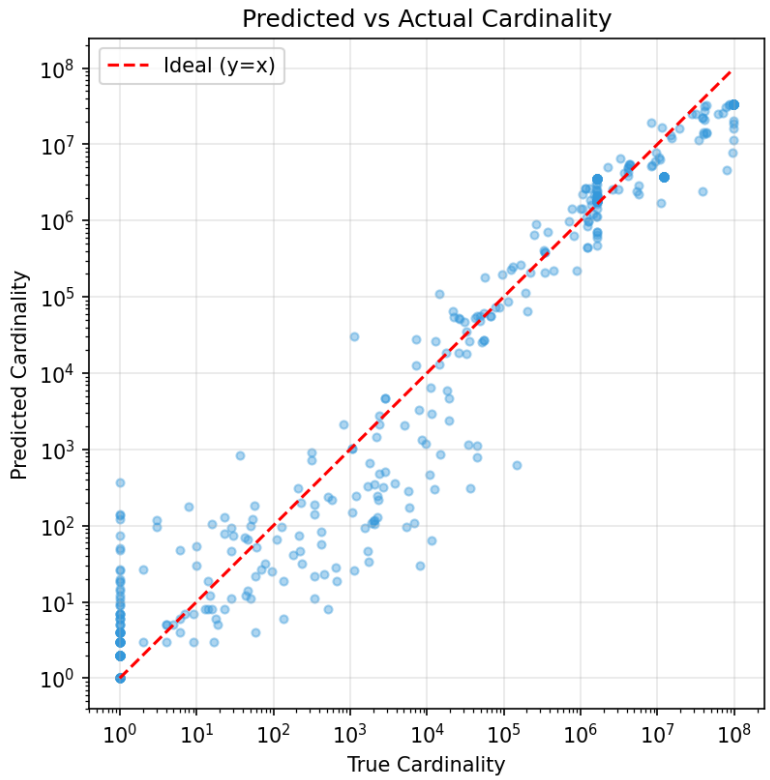

Figure 7.10: Predicted vs. actual cardinality on a log-log scale. The concentration of points along the diagonal indicates that the MSCN model produces well-calibrated estimates across the full cardinality range, from selective single-table queries to large multi-table joins.

Figures 7.11 and 7.12 provide complementary perspectives on the Q-error distribution. The CDF reveals the proportion of queries falling below each error threshold, showing that the majority of predictions are within a small factor of the true cardinality. The per-round box plots illustrate how the error distribution tightens with each active learning round, confirming the progressive improvement in estimation quality.

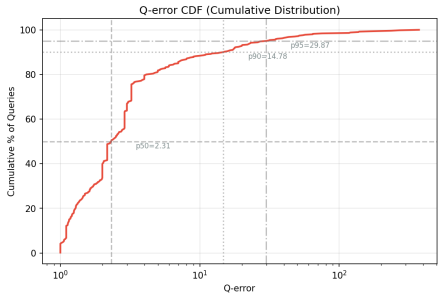

Figure 7.11: Cumulative distribution function of Q-errors. The steep rise at low Q-error values indicates that the majority of queries are estimated within a factor of 2–5 of their true cardinality.

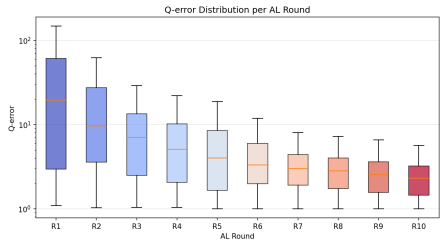

Figure 7.12: Q-error box plots per active learning round. The narrowing boxes and decreasing medians demonstrate how each round improves estimation accuracy.

Figure 7.13 provides a detailed statistical view of Q-error metrics across active learning rounds, showing the median, mean, 90th percentile, and 95th percentile values.

7.4 Labeling Efficiency Analysis

A central claim of this thesis is that active learning reduces the number of expensive database executions needed to train an accurate model. Figure 7.14 directly visualizes this by comparing the labeling effort of the active learning approach against a fully supervised baseline. The results show that the active learning pipeline achieves comparable model accuracy while labeling only a fraction of the total query pool.

Figure 7.15 shows the labeling success rate across the pipeline, indicating what fraction of queries were successfully labeled within the timeout constraints.

7.5 Comparison with PostgreSQL Optimizer

To contextualize the MSCN model's accuracy, we compare its cardinality estimates directly against those produced by the PostgreSQL native query optimizer. Figures 7.16 and 7.17 present this comparison using the same validation queries.

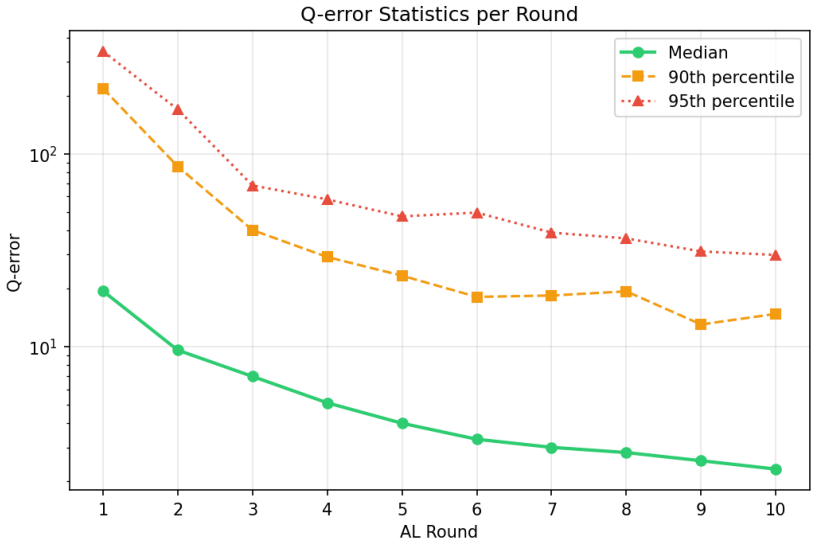

Figure 7.13: Q-error statistics across active learning rounds. The consistent decrease in all percentile metrics confirms that active query selection improves not only typical-case accuracy but also tail-end performance on the most challenging queries.

7.6 Pipeline Summary and Discussion

Figure 7.18 provides a comprehensive summary dashboard of the full pipeline run, combining key metrics from query generation, labeling, training, and evaluation.

The experimental results demonstrate several important findings. The schema-aware validation pipeline effectively filters invalid LLM-generated queries, with the majority passing all validation layers. The structural feature distributions confirm that the diversity hint rotation mechanism successfully prevents mode collapse during generation, producing a balanced mix of query complexities.

The generated workload covers cardinalities spanning multiple orders of magnitude, which is essential for training a generalizable cardinality estimator. The active learning curves show that model accuracy improves steadily with each round, confirming that intelligent query selection substantially reduces the number of expensive database executions required for training.

The predicted versus actual scatter plot demonstrates good model calibration across the full cardinality range. The training loss curves confirm stable convergence within each round, with each subsequent round benefiting from the expanded training set. The comparison with PostgreSQL's native optimizer provides valuable context for the MSCN model's performance, situating the learned approach relative to the established baseline used in production database systems.

These results validate the end-to-end approach of combining LLM-based query

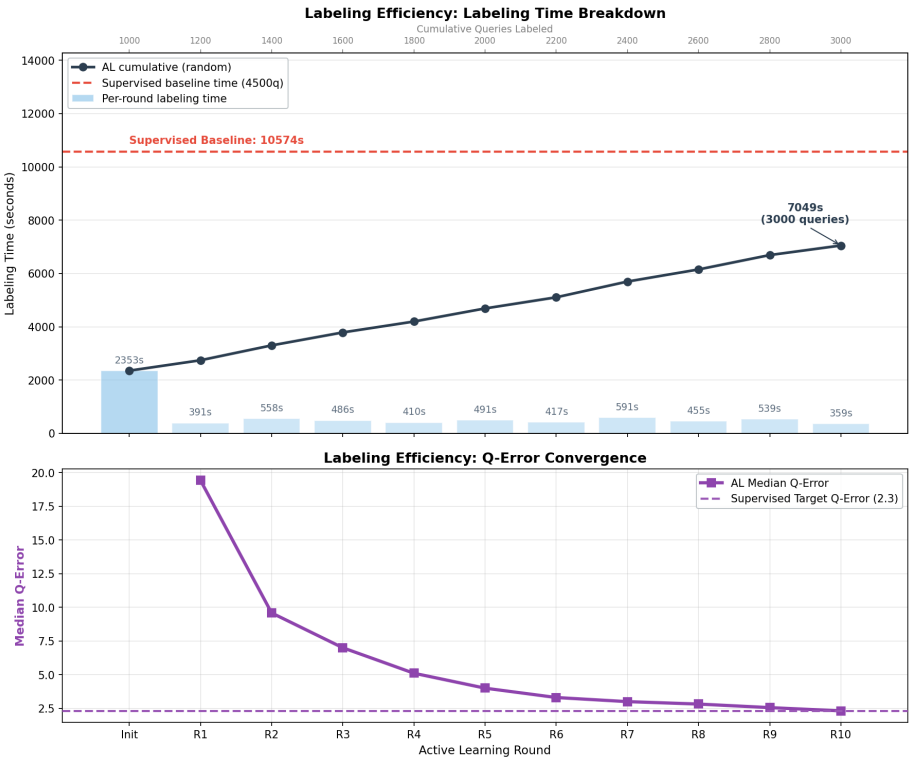

Figure 7.14: Labeling efficiency comparison. The active learning approach achieves competitive accuracy with significantly fewer labeled queries than a fully supervised baseline, demonstrating that intelligent query selection reduces the dominant cost factor in the pipeline.

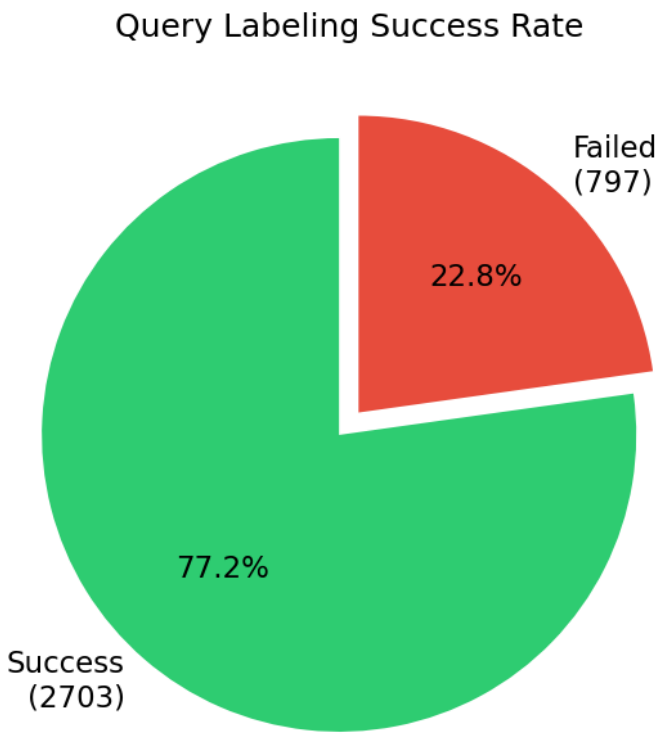

Figure 7.15: Labeling success rate across the pipeline. Queries that exceed the database timeout are assigned fallback cardinalities. The high success rate confirms that the generated queries are overwhelmingly executable within the configured time limits.

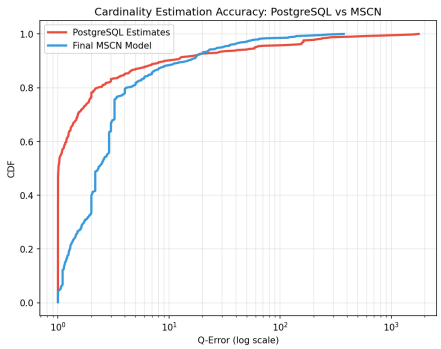

Figure 7.16: Q-error CDF comparing the MSCN model against PostgreSQL's native estimator. A curve shifted further left indicates better estimation accuracy.

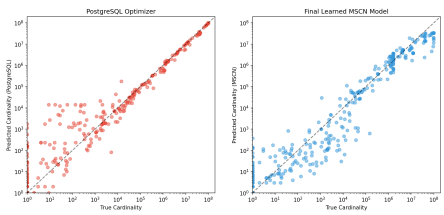

Figure 7.17: Predicted vs. actual scatter comparing MSCN and PostgreSQL. Points closer to the diagonal represent more accurate estimates.

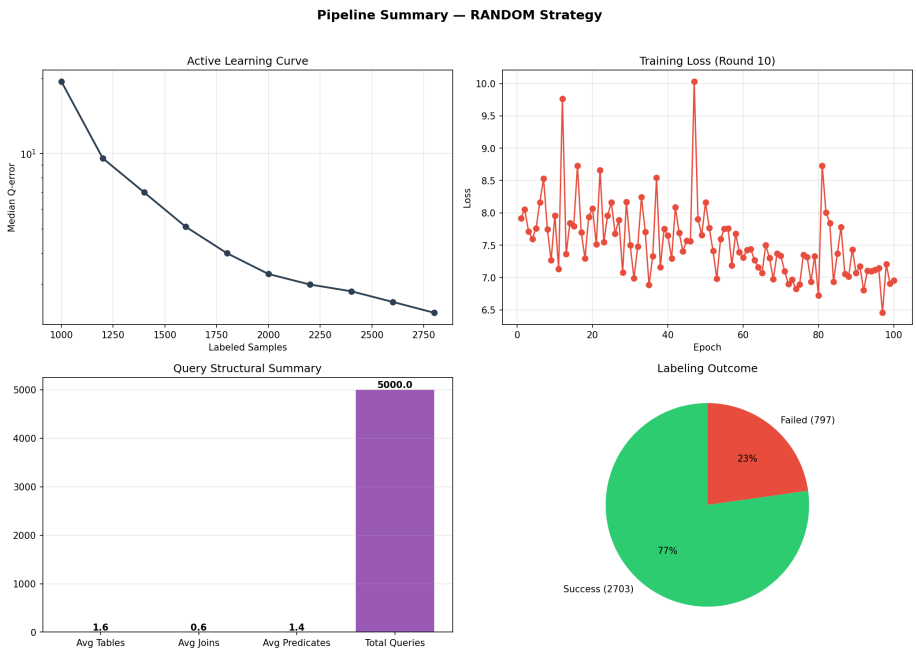

Figure 7.18: Pipeline summary dashboard aggregating key metrics from all stages. This consolidated view enables rapid assessment of the entire experimental run, from data generation quality through model training convergence to final estimation accuracy.

generation with active learning for training learned cardinality estimators, achieving competitive accuracy with a substantially reduced labeling budget.

Chapter 8

Conclusion

8.1 Summary of Contributions

This thesis presented a comprehensive framework for creating and benchmarking learned database components using large language models and active learning. We demonstrated that large language models can generate diverse, realistic SQL workloads at scale, reducing reliance on manual query collection and synthetic templates. By incorporating uncertainty sampling and Monte Carlo dropout strategies into the training process, we achieved notable improvements in sample efficiency compared to random labeling approaches, enabling the model to reach competitive accuracy levels with a substantially smaller labeling budget. The implemented system provides a complete, schema-agnostic solution that integrates query generation, multi-layer validation, database labeling, feature extraction, and model training into a unified and fully automated pipeline. Through comprehensive partitioning strategies including complexity-based, schema-aware, and uncertainty-based methods, the framework enables efficient learning across different workload characteristics. The extensive experimental evaluation on the IMDB dataset established rigorous evaluation protocols using Q-error metrics and learning curves, providing a reliable basis for comparing different training strategies.

8.2 Impact and Implications

The work presented in this thesis advances the field of learned database systems by addressing the critical challenge of training data generation. Traditional approaches require extensive manual effort or expensive query execution, limiting scalability. Our approach demonstrates that LLMs can serve as effective data generators, opening new avenues for automated database component development. From a practical standpoint, the framework enables more efficient training of learned cardinality estimators for better query optimization, facilitates comprehensive testing of database systems through automated generation of diverse workloads, and provides an open platform for researchers to experiment with learned database components. The reduced training costs also make learned approaches more viable for adoption in commercial database systems.

Beyond database systems, the integration of active learning with generative AI represents a broader methodological contribution. It demonstrates the potential of LLMs in generating structured, constraint-satisfying data, shows how uncertainty-based sampling can be combined effectively with modern neural architectures, and provides a template for building automated machine learning pipelines with minimal human intervention.

8.3 Limitations and Future Work

While the framework achieves significant improvements, several limitations remain. The current implementation focuses on relatively simple conjunctive queries; extending to complex analytical queries with subqueries, aggregations, and window functions requires additional research. Although the pipeline is schema-agnostic by design, performance may vary across different database schemas and data distributions. The Monte Carlo dropout inference and bitmap generation introduce computational overhead that may limit real-time applications. Additionally, timeout handling and error recovery during labeling may introduce noise in the training data through fallback cardinality assignments.

Several promising directions for future work emerge from this thesis. On the query generation side, integrating LLMs with other generative models could enable more complex query patterns, and enhancing LLMs with deeper database semantics could improve the quality of generated workloads. Cross-domain transfer methods could enable learned components trained on one schema to be applied to others.

For active learning, multi-objective optimization that balances uncertainty, diversity, and computational cost in acquisition functions represents a natural extension. Adapting the framework for online active learning in streaming query workloads would enable deployment in production systems, and Bayesian optimization methods could provide more sophisticated uncertainty estimation.

From a systems perspective, integrating learned components into existing database systems with minimal overhead, scaling the pipeline for distributed training over large databases, and developing self-tuning systems that adapt to changing workloads automatically are all important directions. On the theoretical side, providing convergence guarantees for active learning in cardinality estimation, establishing sample complexity bounds, and developing generalization theory for learned estimators would strengthen the foundations of this approach.

8.4 Final Remarks

The research presented in this thesis demonstrates the feasibility and benefits of using large language models for automated training data generation in learned database systems. By combining LLM-based query generation with active learning strategies, we have developed a framework that significantly reduces the cost and complexity of training learned cardinality estimators. The empirical results on the IMDB dataset show that our approach achieves competitive accuracy with substantially fewer labeled examples compared to traditional supervised methods.

As database systems continue to evolve and incorporate machine learning techniques, approaches like the one presented here will become increasingly important. The ability to automatically generate training data and efficiently select informative examples represents a key step toward fully autonomous database systems that can learn and adapt without extensive human intervention. The integration of generative AI with traditional database techniques opens exciting

possibilities for the future of data management, and we hope this work inspires further research in this promising direction.

References

[1] A. Davitkova and S. Michel. Automated training of learned database components with generative AI. *CoRR*, abs/2512.20271, 2025.

[2] Y. Gal and Z. Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1050–1059. JMLR.org, 2016.

[3] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper. Learned cardinalities: Estimating correlated joins with deep learning. *CoRR*, abs/1809.00677, 2018.

[4] A. Kipf, T. Kipf, B. Radke, V. Leis, P. Boncz, and A. Kemper. Learned cardinalities: Estimating correlated joins with deep learning. In *9th Biennial Conference on Innovative Data Systems Research, CIDR 2019, Asilomar, CA, USA, January 13-16, 2019, Online Proceedings*. www.cidrdb.org, 2019.

[5] D. Li, Z. Wang, Y. Chen, R. Jiang, W. Ding, and M. Okumura. A survey on deep active learning: Recent advances and new frontiers. *IEEE Trans. Neural Networks Learn. Syst.*, 36(4):5879–5899, 2025.

[6] Q. Liu, B. Chen, J. Guo, M. Ziyadi, Z. Lin, W. Chen, and J. Lou. TAPEX: Table pre-training via learning a neural SQL executor. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.

[7] S. Mansour, L. Perelli, L. Mainetti, G. Davidson, and S. D'Amato. PAARS: Persona aligned agentic retail shoppers. *CoRR*, abs/2503.24228, 2025.

[8] P. Negi, Z. Wu, A. Kipf, N. Tatbul, R. Marcus, S. Madden, T. Kraska, and M. Alizadeh. Robust query driven cardinality estimation under changing workloads. *Proc. VLDB Endow.*, 16(6):1520–1533, 2023.

[9] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Ozcan, and S. Ö. Arik. CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in text-to-SQL. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[10] A. Rahaman, A. Zheng, M. Milani, F. Chiang, and R. Pottinger. Evaluating SQL understanding in large language models. In A. Simitsis, B. Kemme, A. Queralt, O. Romero, and P. Jovanovic, editors, *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*, pages 909–921. OpenProceedings.org, 2025.

[11] T. Schmidt, V. Leis, P. Boncz, and T. Neumann. SQLStorm: Taking database benchmarking into the LLM era. *Proc. VLDB Endow.*, 18(11):4144–4157, 2025.

[12] X. Wang, C. Qu, W. Wu, J. Wang, and Q. Zhou. Are we ready for learned cardinality estimation? *Proc. VLDB Endow.*, 14(9):1640–1654, 2021.